

Module 5 Report

Software Assurance + Secure SQL (SQLi Defense)

Student	Caleb Gevertz (JHED: cgevert1)
Course	Modern Software Concepts in Python, Johns Hopkins University
Assignment	Module 5 - Software Assurance + Secure SQL (SQLi Defense)
Date	2026-10-01
Repository	jhu_software_concepts / module_5

Contents: Fresh Install; Running; Packaging: why setup.py matters; Dependency graph; SQL injection defenses; Least-privilege database; Static analysis: Pylint (10/10); Snyk dependency scan; Continuous integration; Tests.

Fresh Install

Requirements: Python 3.10+ (3.12 is what CI uses), PostgreSQL 14+, and — only to regenerate the dependency graph — Graphviz (<https://graphviz.org/download/>) so that the dot command is on your PATH. Run everything below from the module_5/ folder. requirements.txt pins **every** package (runtime, tooling and their transitive dependencies), so either method rebuilds the identical environment.

Option 1 — pip + venv

```
python -m venv .venv
source .venv/bin/activate          # Windows PowerShell: .venv\Scripts\Activate.ps1
python -m pip install -r requirements.txt
python -m pip install -e .         # puts src/ on the import path (see setup.py)
```

Option 2 — uv

```
uv venv                            # creates .venv
source .venv/bin/activate          # Windows PowerShell: .venv\Scripts\Activate.ps1
uv pip sync requirements.txt        # installs exactly what is listed -- nothing more, nothing less
uv pip install -e .
```

uv pip sync makes the environment match requirements.txt exactly (it also removes anything extra), which is why the file is fully pinned. The short list of direct dependencies lives in requirements.in; regenerate the pinned file after editing it with:

```
uv pip compile requirements.in --universal --python-version 3.12 -o requirements.txt
```

Both methods were verified in brand-new environments: the app imports and serves its page from a different working directory, pip check reports no broken requirements, and a non-editable wheel (pip wheel . --no-deps) also carries the templates and CSS.

Configure the database connection

Credentials are never in the code. Copy the template and fill in real values (.env is git-ignored):

```
cp .env.example .env
```

Variable	Used by	Meaning
DB_HOST, DB_PORT, DB_NAME	app, CLI tools	Server location and database (defaults localhost, 5432, gradcafe; this module uses gradcafe_m5)
DB_USER, DB_PASSWORD	app, CLI tools	The least-privilege role gradcafe_m5_app
DB_OWNER_USER, DB_OWNER_PASSWORD	setup_db.* only	The owner role that creates the schema
TEST_DATABASE_URL	tests	Owner-level URL of the scratch database gradcafe_m5_test
TEST_APP_DATABASE_URL	privilege tests	The app role's URL for the same scratch database
DATABASE_URL	optional	A full URL that replaces the five DB_* values (CI uses it)

Real environment variables always override .env. The old PG* variables are intentionally ignored.

Create the database roles (one time)

src/setup_db.sh (or double-click src/setup_db.bat on Windows) runs src/db_setup.sql as the PostgreSQL superuser and prompts for that password (it is never stored). It reads the two role passwords from .env. See *Least-privilege database* below for exactly what it creates.

Running

```
# load the scraped data (schema must exist: see "Create the database roles")
python src/load_data.py          # add --reset to empty the table first (needs the owner
                                # role)
python src/query_data.py         # print the analysis answers using the composed-SQL layer
python src/orm_queries.py        # the same answers through SQLAlchemy

# the Flask app
cd src && flask --app flask_app run    # http://127.0.0.1:5000/analysis
```

Routes: GET /analysis, POST /pull-data, POST /update-analysis, GET /status, and the new read-only GET /applicants?term=&status=&sort=&direction=&limit= (see *SQL injection defenses*). Example: `curl "http://127.0.0.1:5000/applicants?term=fall%202026&status=accepted&sort=gpa&direction=desc&limit=5"`.

Packaging: why setup.py matters

setup.py makes the project an installable package instead of a folder of scripts. **Why packaging matters:**

- **Imports behave the same everywhere.** The modules in src/ import each other by bare name (import query_data). `pip install -e .` puts src/ on the import path once, so a local shell, pytest and GitHub Actions resolve imports identically, with no hand-set PYTHONPATH and no dependence on which directory you launched from. (This was checked by importing and rendering the app from an unrelated folder.)
- **Editable installs remove "it works on my machine" path bugs.** `pip install -e .` links to the source instead of copying it, so edits take effect immediately and the installed project and the working tree cannot drift apart.
- **Dependencies are declared once, next to the code.** `install_requires` lists what the app needs and `extras_require["dev"]` lists the test, lint and security tooling. Tools like uv can extract those lists when syncing environments, and `requirements.txt` pins the exact tested versions of the same set.
- **A real wheel is complete.** The templates and CSS live in the web_assets package and are listed in package_data, so `pip install .` (not just `-e`) produces a working app; a built wheel was installed into a clean environment to confirm it renders the page.

tests/test_packaging.py keeps setup.py, the imports in src/ and requirements.txt consistent.

Dependency graph

dependency.svg is generated with pydeps and Graphviz. From module_5/, with dot on your PATH:

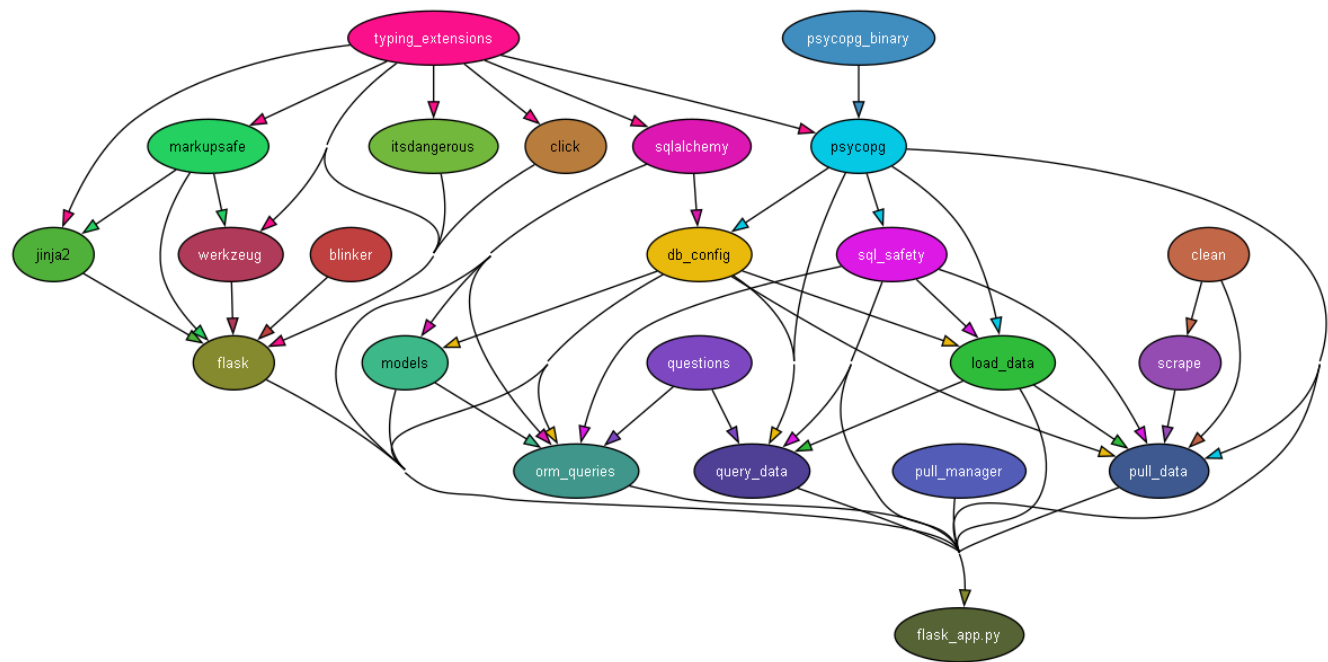
```
pydeps src/flask_app.py --noshow -T svg -o dependency.svg --max-module-depth=1
```

--max-module-depth=1 collapses each third-party package (Flask, psycopg, SQLAlchemy) into a single node so the picture shows how *this project's* modules relate instead of hundreds of Flask internals. pydeps draws each arrow from a dependency to the module that imports it. CI regenerates the file on every push and fails if it is missing or empty.

Key dependencies, in 7 sentences. flask_app.py is the entry point and sits at the bottom of the graph because nearly everything flows into it. Flask and its helpers (Werkzeug for requests and routing, Jinja2 for the page template, plus click, itsdangerous, markupsafe and blinker) form the web layer. Data access has two parallel paths: psycopg (with its compiled psycopg_binary driver) serves the raw-SQL modules query_data, load_data and pull_data, while SQLAlchemy serves models and orm_queries, which feed the analysis page. db_config is the single place both paths read the DB_* connection settings from the environment, so five modules import it and no credentials appear anywhere else. The new sql_safety module (built on psycopg.sql) is imported by query_data, load_data, pull_data and orm_queries, which is how one clamp-and-bind LIMIT rule covers every query. questions holds the shared wording and formatting so the SQL and ORM versions give identical

answers, while clean and scrape feed pull_data (the Pull Data button) under the supervision of pull_manager. Underneath it all, typing_extensions is a shared low-level dependency of SQLAlchemy, pycpg and Flask's helpers, and there are no cycles: dependencies flow in one direction toward the app.

Evidence: Dependency graph (rendered from dependency.svg)



SQL injection defenses

Rule: SQL text is composed with pycpg.sql; values only ever travel as bound parameters.

Module 4 (before)	Module 5 (after)
load_data.UPSERT_SQL built with an f-string and ", ".join(COLUMNS)	UPSERT_STATEMENT composed from sql.SQL, sql.Identifier (every column) and sql.Placeholder (every value)
Eleven analysis queries as raw strings with the filter values written into the text (... = 'fall 2026', ILIKE 'accept%')	query_data.py composes each from sql.SQL; table is sql.Identifier("applicants"); every filter value is a named placeholder %(term)s, %(accepted)s, ...) bound from a dict
SELECT p_id FROM applicants read the whole table	Keyset-paginated, LIMIT-ed pages (pull_data._known_ids)
fetch_applicants(limit=None) returned every row	Always capped; the default is the maximum (100)
No endpoint accepted user input	New read-only GET /applicants?term&status&sort&direction&limit, built on query_data.search_applicants

Construction is separate from execution. Every query follows the same two steps:

```
statement = limited(query.statement) # build: a composed SQL object + "LIMIT %(limit)s"
cursor.execute(statement, limit_params(limit, query.params)) # run: statement and parameters apart
```

What each kind of variable part does

- *Values* (term, status, LIKE and regex patterns, the LIMIT): `sql.Placeholder /%(name)s`, filled from a parameter dict. term and status are compared with `=`, not LIKE, so a user's `%` or `_` is an ordinary character and cannot widen the match.
- *Identifiers* (table and column names): `sql.Identifier`. The only user-influenced identifier is the sort column. It must exactly equal one of the allowed column names, and it only *selects* a pre-built `sql.Identifier` from `SORT_IDENTIFIERS`; the request's text is never copied into the SQL.
- *Direction* (ASC/DESC) is chosen from a boolean in code, never from request text.

LIMIT on every query (src/sql_safety.py)

- `limited(stmt)` appends `LIMIT %(limit)s` to every SELECT; the ORM path calls `.limit()` on every select. A test records every statement sent to PostgreSQL (raw `psycopg` and `SQLAlchemy`) and fails if any SELECT lacks a LIMIT.
- `clamp_limit()` keeps the value in **1..100** (`MIN_LIMIT..MAX_LIMIT`): `limit=99999999` becomes 100, 0 and negatives become 1, and no limit means the default of 100. Something that is not a whole number (`abc, 1; DROP TABLE applicants, 2.5, 1e3`) raises `ValueError`, which the endpoint turns into 400 Bad Request with a fixed message that never echoes the input.
- Write statements (`INSERT ... ON CONFLICT, TRUNCATE, CREATE TABLE`) return no rows, so LIMIT does not apply to them; they carry only bound values (the upsert) or no values at all.

Hostile input is tested, not assumed. `tests/test_applicants_endpoint.py`, `tests/test_query_data.py` and `tests/test_sql_safety.py` send payloads such as `' OR '1'='1, '; DROP TABLE applicants; --, %, p_id; DROP TABLE applicants --` and `pg_sleep(5)` as values, sort columns, directions and limits, and check there is no crash, no extra rows, no leaked data and that the table is still intact.

`tests/test_sql_composition_guard.py` parses every file in `src/` and fails if any `execute()` / `executemany()` is ever given an f-string, `+/%` concatenation, `.format()` or string literal.

Least-privilege database

The web app should hold only the database power it needs. A superuser, or even the owner of the tables, could drop or rewrite everything if an injection ever got through. So `src/db_setup.sql` creates two **non-superuser** roles (neither can create roles or databases, replicate, or bypass row security):

Role	Purpose	Holds
<code>gradcafe_m5_owner</code>	Owns the databases and the applicants table. Used only to provision the schema, for <code>load_data.py --reset</code> , and by the test suite on its scratch database.	Ownership (DDL)
<code>gradcafe_m5_app</code>	What the Flask app and the command-line tools log in as.	The grants below, and nothing else

Permissions granted to `gradcafe_m5_app`, and why

Permission	Why the app needs it
CONNECT on <code>gradcafe_m5</code> (and the scratch database)	To log in. PUBLIC has no CONNECT, so no other role can.
USAGE on schema <code>public</code>	To reach the table. There is deliberately no CREATE on the schema.
SELECT on <code>applicants</code>	Every analysis query, <code>GET /applicants</code> , and the "which ids do I already have?" read before a pull.

Permission	Why the app needs it
INSERT on applicants	The Pull Data button and <code>load_data.py</code> add new rows.
UPDATE (<code>llm_generated_program</code> , <code>llm_generated_university</code>)	The upsert's ON CONFLICT ... DO UPDATE may fill an <i>empty</i> LLM column. Column-level, so no other column can be modified.

Not granted: DELETE, TRUNCATE, REFERENCES, TRIGGER, DROP, ALTER, CREATE, or ownership of anything. The role cannot hand out privileges either.

The SQL (`src/db_privileges.sql`, run once per database by `db_setup.sql`):

```
SET ROLE gradcafe_m5_owner; -- the table is created and owned by the owner
CREATE TABLE IF NOT EXISTS applicants ( ... );
REVOKE ALL ON TABLE applicants FROM PUBLIC;
REVOKE ALL ON TABLE applicants FROM gradcafe_m5_app;
GRANT SELECT, INSERT ON TABLE applicants TO gradcafe_m5_app;
GRANT UPDATE (llm_generated_program, llm_generated_university)
    ON TABLE applicants TO gradcafe_m5_app;
RESET ROLE;
REVOKE ALL ON SCHEMA public FROM PUBLIC;
GRANT USAGE ON SCHEMA public TO gradcafe_m5_app; -- usage only: no CREATE
```

Credentials come from the environment. The roles' passwords are read from `DB_OWNER_PASSWORD` and `DB_PASSWORD` (`\getenv` in `db_setup.sql`), never written into a script. The application reads `DB_HOST`, `DB_PORT`, `DB_NAME`, `DB_USER`, `DB_PASSWORD` (see *Configure the database connection*). `.env.example` has placeholders only, and `.env` is git-ignored.

Provision and verify (from `module_5/src/`; `psql` prompts for the postgres password):

```
./setup_db.sh # Windows: double-click setup_db.bat
psql -U gradcafe_m5_owner -h localhost -d gradcafe_m5 -f db_verify_privileges.sql # privileges
    held
psql -U gradcafe_m5_app -h localhost -d gradcafe_m5_test -f db_demo_refusals.sql # forbidden
    statements
```

The owner role is needed only for schema changes. For example, to empty and reload the table:

```
DB_USER=gradcafe_m5_owner DB_PASSWORD=<owner password> python src/load_data.py --reset.
```

It is tested, not just configured. `tests/test_db_setup_script.py` reviews the SQL (no password literals, no destructive grant, UPDATE limited to the two columns). `tests/test_least_privilege.py` logs in as the real application role and checks that DROP, ALTER, TRUNCATE, DELETE, CREATE TABLE, CREATE ROLE, updating any other column and granting to PUBLIC all fail, that the data survives, and that the whole app (analysis page, /applicants, Update Analysis, Pull Data) still works under those limits. The Flask pull and the loader no longer run DDL; a recorder test fails if either ever sends any.

Evidence: Least-privilege evidence: psql output

Least-privilege evidence for Module 5

Captured: 2026-10-01 23:25:42 EDT

Server: PostgreSQL 18.6 on x86_64-windows

1. psql -U gradcafe_m5_owner -d gradcafe_m5 -f db_verify_privileges.sql

--- 1. Role attributes: neither role is a superuser and neither can create roles or databases

rolname	rolsuper	rolcreatedb	rolcreatorole	rolreplication	rolbypassrls
gradcafe_m5_app	f	f	f	f	f
gradcafe_m5_owner	f	f	f	f	f

(2 rows)

--- 2. Table-level privileges held by gradcafe_m5_app on applicants (expect SELECT, INSERT)
privilege_type

INSERT
SELECT
(2 rows)

--- 3. Which columns can gradcafe_m5_app UPDATE (expect only the two llm_* columns)
column_name

llm_generated_program
llm_generated_university
(2 rows)

--- 4. Owner-only operations (expect f for every row)

privilege | app_has_it

-----+-----
DELETE | f
TRUNCATE | f
REFERENCES | f
TRIGGER | f
(4 rows)

privilege | app_has_it
-----+-----
CREATE on schema public | f
(1 row)

privilege | app_has_it
-----+-----
owns the applicants table | f
(1 row)

2. psql -U gradcafe_m5_app -d gradcafe_m5_test -f db_demo_refusals.sql

Pager usage is off.

connected_as	database
gradcafe_m5_app	gradcafe_m5_test

(1 row)

>>> DROP TABLE applicants

BEGIN

psql:db_demo_refusals.sql:17: ERROR: must be owner of table applicants

ROLLBACK

>>> ALTER TABLE applicants ADD COLUMN extra TEXT

BEGIN

psql:db_demo_refusals.sql:20: ERROR: must be owner of table applicants

ROLLBACK

>>> TRUNCATE applicants

BEGIN

psql:db_demo_refusals.sql:23: ERROR: permission denied for table applicants

ROLLBACK

>>> DELETE FROM applicants

BEGIN

psql:db_demo_refusals.sql:26: ERROR: permission denied for table applicants

ROLLBACK

>>> UPDATE applicants SET status = 'hacked' (only the two llm_* columns are updatable)

```

BEGIN
psql:db_demo_refusals.sql:29: ERROR:  permission denied for table applicants
ROLLBACK
>>> CREATE TABLE evil (id INT)
BEGIN
psql:db_demo_refusals.sql:32: ERROR:  permission denied for schema public
LINE 1: CREATE TABLE evil (id INT);
                        ^
ROLLBACK
>>> CREATE ROLE evil LOGIN
BEGIN
psql:db_demo_refusals.sql:35: ERROR:  permission denied to create role
DETAIL:  Only roles with the CREATEROLE attribute may create roles.
ROLLBACK

>>> SELECT count(*) FROM applicants    (allowed)
      rows_visible
-----
              1
(1 row)

```

Static analysis: Pylint (10/10)

Pylint runs on every Python file under `src/` and nothing outside it. From `module_5/` with the virtual environment active:

```
pylint src
```

Expected final line: Your code has been rated at 10.00/10. The saved output is in `pylint_report.txt`. CI runs `pylint src --fail-under=10`, so any score below 10 fails the build.

Fixes were made in the code wherever possible (docstrings, shared helpers instead of duplicated blocks, `functools.lru_cache` instead of a global, a context manager instead of a bare `open`). The few remaining inline `# pylint: disable=...` comments each carry a one-line justification: SQLAlchemy's dynamic func namespace (a known not-callable false positive), the job-runner boundary that must catch every exception, declarative models with no methods, the dependency-injection `create_app` factory, and two optional LLM packages that are intentionally not installed.

Evidence: Pylint output

```

# Pylint evidence for module_5/src (2026-10-01)
# Command (run from module_5/, venv active):  pylint src
# Version: pylint 4.1.1

```

```

-----
Your code has been rated at 10.00/10 (previous run: 10.00/10, +0.00)

```

Snyk dependency scan

Supply-chain check of the pinned dependencies in `requirements.txt`. With the virtual environment active (Snyk's Python plugin reads the *installed* packages) and after a one-time `snyk auth`:

```
snyk test --file=requirements.txt --package-manager=pip
```

The output is saved as the screenshot `snyk-analysis.png`. Extra credit, static analysis of the source:

```
snyk code test
```

with the evidence in `snyk-code-analysis.png`. Any vulnerability Snyk reports is documented below with the action taken (upgrade the pinned version in `requirements.in`, regenerate `requirements.txt`, or remove the package).

Findings (2026-10-01). `snyk test` tested all 37 pinned dependencies and reported **0 issues** ("no vulnerable paths found"), so no package had to be patched or removed. The full output is saved in `snyk_test_output.txt`. `snyk-analysis.png` is a terminal-style rendering of that exact saved output, captioned as such in the image itself.

The pins are re-checked on every push by the CI snyk job (the SNYK_TOKEN secret is set, and run #7 performed the real scan and passed), which would flag any high or critical vulnerability disclosed after this date.

Snyk Code (extra credit, snyk code test src). The first scan found 9 issues: one **HIGH** SQL Injection and eight **LOW** Path Traversal notes. Evidence: snyk_code_output.txt and snyk-code-analysis.png (a captioned rendering of the saved output of the final scan).

- *HIGH, flask_app.py line 136 (GET /applicants into sql_safety.limited).* The sort parameter was already checked against an allow-list and quoted with sql.Identifier, so this was not exploitable, but Snyk could not see the guard. To find out which parameter it was following, each request-derived argument was replaced by a constant in a scratch copy and rescanned: only sort made the finding disappear. Snyk follows a dictionary lookup by a tainted key, so the code now finds the trusted identifier by *comparing* sort with each allowed column name and returning the pre-built constant (query_data._trusted_sort_identifier). The request text is never copied into the SQL, which is also the stronger design. The term/status conditions and the ASC/DESC keyword are likewise pre-built constants. The rescan reports **0 HIGH, 0 MEDIUM**; the existing injection tests (and new ones for non-string sort values) all pass.
- *LOW, Path Traversal x8 (clean.py, scrape.py, load_data.py, llm_hosting/app.py).* These are command-line tools whose --input / --output / --data arguments are file paths typed by the person running the tool, who already has that file access; no web route accepts a path. Confining them would only break legitimate use (for example loading data from another folder), so they are an accepted, documented risk and the code is unchanged.

Evidence: snyk test (snyk-analysis.png)

```
PS C:\Users\Caleb\jhu_software_concepts\module_5> snyk test --file=requirements.txt --package-manager=pip
Testing (requirements.txt) ...
```

✓ Tested 37 dependencies for known issues, no vulnerable paths found.

Test Summary

Organization:	caleb-gvz
Test type:	open-source
Project path:	C:\Users\Caleb\jhu_software_concepts\module_5
Total issues:	0

💡 Tip

To view ignored issues, use the --include-ignores option.

Rendered from the saved output of the command above (snyk_test_output.txt), run 2026-10-01.

Evidence: snyk code test (extra credit)

```
PS C:\Users\Caleb\jhu_software_concepts\module_5> snyk code test src
Testing src ...

Open Issues

X [LOW] Path Traversal
Finding ID: b29a037d-22b0-4ec8-8400-3947872d7301
Path: clean.py, line 77
Info: Unsantitized input from a command line argument flows into open, where it is used as a path. This may result in a Path Traversal
vulnerability and allow an attacker to read arbitrary files.

X [LOW] Path Traversal
Finding ID: ad6cd7d8-42da-4846-8f09-1fc9dcf7375c
Path: llm_hosting/app.py, line 302
Info: Unsantitized input from a command line argument flows into open, where it is used as a path. This may result in a Path Traversal
vulnerability and allow an attacker to read arbitrary files.

X [LOW] Path Traversal
Finding ID: ba698d41-48f0-4e89-8fb8-b8a0bd0dccc3f
Path: load_data.py, line 343
Info: Unsantitized input from a command line argument flows into open, where it is used as a path. This may result in a Path Traversal
vulnerability and allow an attacker to read arbitrary files.

X [LOW] Path Traversal
Finding ID: 96101d3c-7775-4cf1-8ff5-db49123c8b0e
Path: scrape.py, line 211
Info: Unsantitized input from a command line argument flows into open, where it is used as a path. This may result in a Path Traversal
vulnerability and allow an attacker to read arbitrary files.

X [LOW] Path Traversal
Finding ID: 6f298db9-da60-40d5-aadd-780296b2e788
Path: scrape.py, line 218
Info: Unsantitized input from a command line argument flows into open, where it is used as a path. This may result in a Path Traversal
vulnerability and allow an attacker to read arbitrary files.

X [LOW] Path Traversal
Finding ID: 52e3daa1-31d7-489b-822a-028ef48c3b9f
Path: clean.py, line 84
Info: Unsantitized input from a command line argument flows into json.dump, where it is used as a path. This may result in a Path
Traversal vulnerability and allow an attacker to write arbitrary files.

X [LOW] Path Traversal
Finding ID: 4d4e2c97-d4f4-400a-8b21-291ad3ffebc9
Path: scrape.py, line 204
Info: Unsantitized input from a command line argument flows into json.dump, where it is used as a path. This may result in a Path
Traversal vulnerability and allow an attacker to write arbitrary files.

X [LOW] Path Traversal
Finding ID: 16f69021-c207-4af1-87d5-96bb464424f9
Path: scrape.py, line 224
Info: Unsantitized input from a command line argument flows into json.dump, where it is used as a path. This may result in a Path
Traversal vulnerability and allow an attacker to write arbitrary files.

Test Summary
Organization:      caleb-gvz
Test type:         Static code analysis
Project path:      src

Total issues:      8
Ignored issues:    0 [ 0 HIGH 0 MEDIUM 0 LOW ]
Open issues:       8 [ 0 HIGH 0 MEDIUM 8 LOW ]

Tip

To view ignored issues, use the --include-ignores option.

Rendered from the saved output of the command above (snyk_code_output.txt), run 2026-10-01.
```

Continuous integration

`.github/workflows/ci.yml` (at the repository root, next to `module_5/`) runs on **every push and pull request** as four separate jobs, so each failure is visible on its own:

Job	What it enforces
pylint	<code>pylint src --fail-under=10</code> : the build fails if the score is below 10.
dependency-graph	Installs Graphviz, runs <code>pydeps ... -o dependency.svg</code> , and <code>test -s dependency.svg</code> fails the job if the file is missing or empty. The SVG is uploaded as an artifact.

Job	What it enforces
snyk	snyk test on the pinned dependencies, failing on high or critical issues. It reads the SNYK_TOKEN repository secret (set); without one the job would pass with a visible warning that the scan was skipped.
pytest	Starts PostgreSQL 16, runs src/db_setup.sql to create the owner and least-privilege roles, then runs the full marked suite with the 100% coverage gate (--cov-fail-under=100 in pytest.ini), including the live least-privilege tests. A failing test fails the build.

All four jobs install from the pinned requirements.txt. Module 4's workflow (tests.yml) keeps running for module_4/. A screenshot of a successful run is actions_success.png.

The Snyk job needs a repository secret named SNYK_TOKEN (GitHub: *Settings > Secrets and variables > Actions > New repository secret*) holding a Snyk Personal Access Token (Snyk account settings > Personal access tokens; they expire after at most 90 days, so the secret must be renewed). The screenshot above is run #7, the first run with the secret in place: the real scan step ran and the "skipped" notice step did not.

Evidence: GitHub Actions: a successful run

The screenshot shows the GitHub Actions interface for a workflow named 'module_5 CI'. The workflow is triggered by a push to the 'main' branch. The status is 'Success' with a total duration of '1m 7s' and one artifact. The workflow file 'ci.yml' is shown, listing the jobs: pylint (36s), dependency-graph (37s), snyk (27s), and pytest (1m 4s). The 'snyk' job is highlighted, showing a warning icon and the text '4 warnings and 4 notices'.

Tests

pytest -m "web or buttons or analysis or db or integration"

pytest.ini enforces 100% coverage of src/ (--cov-fail-under=100); the same command works from the repository root as pytest module_5 -m "...". The final summary is saved in coverage_summary.txt. Tests

that need the least-privilege role are skipped, with a stated reason, unless TEST_APP_DATABASE_URL is set.

Evidence: Test and coverage summary

```
===== tests coverage =====
_____ coverage: platform win32, python 3.14.3-final-0 _____

Name                               Stmts  Miss  Cover   Missing
-----
src\clean.py                       55      0  100%
src\db_config.py                   53      0  100%
src\flask_app.py                   104      0  100%
src\formatting.py                  19      0  100%
src\llm_hosting\app.py            150      0  100%
src\load_data.py                   190      0  100%
src\make_report.py                 207      0  100%
src\make_zip.py                    56      0  100%
src\models.py                      37      0  100%
src\orm_queries.py                 79      0  100%
src\pull_data.py                   75      0  100%
src\pull_manager.py                65      0  100%
src\query_data.py                  121      0  100%
src\questions.py                   57      0  100%
src\run_llm_standardization.py     83      0  100%
src\scrape.py                      212      0  100%
src\sql_safety.py                  25      0  100%
src\web_assets\__init__.py         0      0  100%
-----
TOTAL                             1588      0  100%
Required test coverage of 100% reached. Total coverage: 100.00%
417 passed in 27.61s
```